Regresión Logística bajo Datos Faltantes: Origen de los NaN, riesgos numéricos, estrategias de imputación y complementariedad con simulación Monte Carlo

Dr. Alberto Muñoz

24 de marzo de 2026

Resumen

La regresión logística es uno de los modelos más importantes para clasificación binaria por su simplicidad, interpretabilidad y bajo costo computacional. Sin embargo, en entornos reales, especialmente industriales, clínicos y financieros, rara vez se dispone de datos completamente limpios. Es frecuente encontrar valores faltantes representados como `NaN`, lo que afecta tanto el entrenamiento como la inferencia del modelo. Este documento técnico presenta una explicación detallada de la regresión logística en presencia de datos incompletos. Se discute de dónde surgen los `NaN`, qué implicaciones tienen sobre la estimación de parámetros, qué otros problemas numéricos y estadísticos pueden aparecer, y cómo distintas estrategias de imputación pueden restaurar la utilidad predictiva del modelo. Además, se explica cómo la simulación Monte Carlo puede complementar el análisis al cuantificar incertidumbre, evaluar sensibilidad ante distintos esquemas de imputación y construir escenarios robustos para toma de decisiones. El objetivo es presentar un marco metodológico riguroso, interpretable y accionable en modo white paper.

1. Introducción

La regresión logística se utiliza para modelar la probabilidad de ocurrencia de un evento binario, por ejemplo, defecto/no defecto, fraude/no fraude, falla/no falla, aceptación/rechazo o enfermedad/no enfermedad. A pesar de su nombre, no es una regresión en el sentido clásico de predecir una variable continua, sino un modelo probabilístico para clasificación. Su popularidad proviene de tres propiedades fundamentales: produce probabilidades interpretables, su frontera de decisión es clara y sus coeficientes admiten lectura en términos de *log-odds*.

No obstante, la calidad del modelo depende críticamente de la calidad del dato. En un entorno real, sensores defectuosos, errores de captura, retrasos de integración, problemas de comunicación, registros parciales, cambios de unidad, fallas de ETL o filtros mal implementados introducen datos faltantes. En muchas bibliotecas numéricas y lenguajes como Python, dichos faltantes

suelen representarse como `NaN`, abreviatura de *Not a Number*. Aunque a primera vista parezca una simple marca de ausencia, su impacto es profundo: muchos algoritmos no pueden entrenar directamente con `NaN`, otros producen resultados sesgados si se ignora el problema, y algunos incluso fallan numéricamente.

En consecuencia, el problema no consiste únicamente en "rellenar huecos", sino en diseñar una estrategia estadística y computacional coherente que preserve la señal relevante, no introduzca sesgos evitables y mantenga la trazabilidad del proceso. Este white paper aborda ese problema desde la óptica de la regresión logística y su posible fortalecimiento mediante técnicas de simulación Monte Carlo.

2. Fundamentos de la regresión logística

2.1. Definición del modelo

Sea un vector de entrada $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)^T$. La regresión logística modela la probabilidad de la clase positiva como

$$P(Y = 1 \mid \boldsymbol{x}) = \pi(\boldsymbol{x}) = \frac{1}{1 + \exp(-z)},$$

donde

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p.$$

La función logística o sigmoide transforma un valor real arbitrario en un valor entre 0 y 1. Esto permite interpretar la salida como probabilidad.

Equivalentemente, el modelo puede escribirse en términos del *logit*:

$$\log\left(\frac{\pi(\boldsymbol{x})}{1 - \pi(\boldsymbol{x})}\right) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j.$$

Esta ecuación muestra que la regresión logística es lineal en el espacio de los *log-odds*, no necesariamente en el espacio de probabilidades.

2.2. Estimación de parámetros

Dado un conjunto de observaciones $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{0, 1\}$, la verosimilitud es

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \pi(\boldsymbol{x}_i)^{y_i} [1 - \pi(\boldsymbol{x}_i)]^{1-y_i}.$$

Su log-verosimilitud es

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[y_i \log \pi(\boldsymbol{x}_i) + (1 - y_i) \log(1 - \pi(\boldsymbol{x}_i)) \right].$$

En la práctica se maximiza $\ell(\boldsymbol{\beta})$ o, equivalentemente, se minimiza la pérdida logística negativa:

$$J(\boldsymbol{\beta}) = -\ell(\boldsymbol{\beta}).$$

Esta optimización se realiza mediante métodos iterativos como gradiente descendente, Newton-Raphson, IRLS o variantes cuasi-Newton como LBFGS.

2.3. Interpretación

Cada coeficiente β_j representa el cambio en el logaritmo de las odds al incrementar una unidad la variable x_j, manteniendo las demás constantes. En términos multiplicativos,

$$\exp(\beta_j)$$

indica el factor por el cual se multiplican las odds cuando x_j aumenta en una unidad.

Esta interpretabilidad es una gran ventaja frente a modelos más opacos, pero también hace evidente el peligro de alimentar al modelo con datos mal imputados: un sesgo en la imputación puede propagarse como un sesgo en las odds estimadas.

3. ¿De dónde salen los NaN?

3.1. Origen computacional

Un `NaN` puede aparecer explícitamente porque el dato no existe o no fue capturado. Por ejemplo, un sensor de temperatura deja de transmitir y el sistema registra un valor vacío. También puede surgir implícitamente por operaciones inválidas, tales como:

$$\frac{0}{0}, \qquad \infty - \infty, \qquad \log(-1), \qquad \sqrt{-1}$$

en un dominio real. En Python y Numpy, muchas de estas operaciones producen `NaN` o `inf`. Si el pipeline posterior no filtra estos valores, terminan contaminando el conjunto de entrenamiento.

3.2. Origen de negocio o de proceso

Desde la perspectiva de operación, los `NaN` aparecen por múltiples razones:

- sensores descalibrados o intermitentes;
- paquetes de telemetría perdidos;
- campos no obligatorios en formularios;
- retrasos entre sistemas transaccionales y analíticos;
- reglas de negocio que excluyen mediciones en ciertos estados;
- errores humanos de captura;
- fallas en ETL, parsing o joins entre tablas;
- truncamiento por límites de medición o saturación de equipo.

3.3. Mecanismos estadísticos del faltante

No todos los faltantes tienen la misma gravedad. Conviene distinguir tres mecanismos:

3.3.1. MCAR: Missing Completely At Random

Los datos faltan completamente al azar. La probabilidad de ausencia no depende ni del valor faltante ni de otras variables observadas. Este es el caso más benigno. Bajo MCAR, una imputación simple puede ser aceptable si la proporción faltante es baja.

3.3.2. MAR: Missing At Random

La ausencia depende de variables observadas, pero no del valor faltante mismo una vez condicionamos en dichas variables. Por ejemplo, una medición de vibración falta más en ciertas líneas de producción o turnos. Aquí, métodos como regresión, KNN o imputación multivariante pueden explotar esa estructura.

3.3.3. MNAR: Missing Not At Random

La probabilidad de ausencia depende del propio valor faltante. Por ejemplo, un sensor falla precisamente cuando la temperatura supera cierto umbral. Este es el caso más delicado, pues imputar sin modelar el mecanismo de ausencia puede inducir sesgos severos.

4. ¿Qué problemas causan los NaN en regresión logística?

4.1. Imposibilidad de entrenamiento directo

Muchas implementaciones estándar, como `LogisticRegression` de `scikit-learn`, no admiten directamente valores `NaN`. Si el conjunto de entrada contiene faltantes, el entrenamiento se inte-

rrumpe o exige un preprocesamiento previo.

4.2. Pérdida de información por eliminación completa

Una estrategia básica es eliminar filas con algún `NaN`. Esta solución puede funcionar cuando el porcentaje faltante es minúsculo, pero suele ser costosa. Si el faltante es frecuente o está concentrado en variables importantes, la eliminación reduce tamaño muestral, debilita potencia estadística y puede sesgar la muestra final.

4.3. Sesgo en parámetros

Una imputación inapropiada puede alterar medias, varianzas, correlaciones e incluso la separación entre clases. Como consecuencia, los coeficientes β_j estimados dejan de reflejar el proceso real y pasan a reflejar el artefacto de imputación.

4.4. Problemas numéricos

Los `NaN` rara vez vienen solos. Suelen coexistir con otros problemas:

- **Infinito o overflow:** cuando $z = \beta_0 + \boldsymbol{\beta}^T \boldsymbol{x}$ toma magnitudes extremas, $\exp(-z)$ puede desbordarse numéricamente.
- **Separación completa o cuasi-completa:** una o más variables separan casi perfectamente las clases, lo cual hace que los coeficientes crezcan sin límite.
- **Colinealidad:** variables altamente correlacionadas vuelven inestable la estimación.
- **Desbalance de clases:** si la clase positiva es muy rara, accuracy puede verse bien aunque el modelo sea inútil.
- **Escalamiento dispar:** variables con órdenes de magnitud muy diferentes dificultan convergencia.
- **Outliers:** sesgan tanto imputación como estimación logística.
- **Data leakage:** imputar usando información del conjunto de prueba contamina la validación.

5. Cómo puede funcionar la regresión logística con datos faltantes

5.1. Estrategia general

La idea no es pedirle a la regresión logística que "adivine" valores faltantes por sí sola, sino construir un pipeline donde primero se trata el faltante y luego se entrena el clasificador. Una arquitectura razonable es:

Datos crudos → Diagnóstico del faltante → Imputación/Eliminación → Escalamiento → Regresión logística →

5.2. Imputación por media

Para una variable x, si sus valores observados son $\{x_i : x_i \neq \texttt{NaN}\}$, la media imputada es

$$\bar{x} = \frac{1}{n_{\text{obs}}} \sum_{i:x_i \neq \texttt{NaN}} x_i.$$

Todo faltante se reemplaza por $\bar{x}$. Es rápida, simple y útil como baseline. Sin embargo, reduce varianza y puede subestimar incertidumbre.

5.3. Imputación por mediana

La mediana sustituye a cada faltante con el valor central de la distribución observada. Es más robusta ante outliers y suele ser preferible cuando la variable está sesgada o tiene valores extremos. En muchos escenarios industriales, la mediana es una opción conservadora y estable.

5.4. Imputación por regresión

Si una variable faltante se correlaciona con otras, puede entrenarse un modelo auxiliar para predecirla. Por ejemplo, si temperatura depende de tiempo de ciclo, presión y consumo, una regresión puede estimar la temperatura faltante a partir de esos covariables.

Esta estrategia es coherente con el mecanismo MAR y tiende a preservar estructura multivariada mejor que la media o mediana.

5.5. Imputación por KNN

En KNN, cada observación con faltante se completa usando las observaciones más parecidas disponibles. Es especialmente útil cuando la relación entre variables es local y no necesariamente lineal. No obstante, su desempeño depende de la métrica de distancia, el valor de k, el escalamiento previo y la densidad de datos.

5.6. Eliminación selectiva de variables

Si una columna tiene una proporción extremadamente alta de faltantes, puede ser mejor eliminarla. Formalmente, si

$$\frac{\#\text{NaN en la columna}}{\#\text{filas totales}} > \tau,$$

con un umbral τ por ejemplo de 0.5, el costo de mantener esa variable puede superar su beneficio.

5.7. Indicador de ausencia

Una práctica muy útil es agregar una variable binaria que indique si el valor original estaba ausente:

$$m_j = \begin{cases} 1, & \text{si } x_j \text{ estaba faltante} \\ 0, & \text{si } x_j \text{ estaba observado} \end{cases}$$

De esta manera, la regresión logística no solo recibe el valor imputado sino también la información de que hubo una ausencia. Esto puede ser muy valioso si el patrón de falta contiene señal predictiva.

6. Peligros y errores frecuentes

6.1. Imputar antes de separar entrenamiento y prueba

Uno de los errores más comunes es imputar sobre el dataset completo antes de hacer la partición train-test. Eso introduce fuga de información: estadísticas del conjunto de prueba, como media o mediana, terminan influyendo sobre el entrenamiento. La regla correcta es:

1. dividir en entrenamiento y prueba;
2. ajustar el imputador solo con entrenamiento;
3. aplicar ese imputador a entrenamiento y prueba;
4. entrenar la regresión logística;
5. evaluar.

6.2. Pensar que toda imputación mejora

No toda imputación mejora el modelo. Algunas introducen más ruido que señal. Por eso conviene evaluar con métricas como accuracy, precision, recall, F1, ROC-AUC y, si se usa una salida continua complementaria o una tarea auxiliar, RMSE o MAE.

6.3. Confundir conveniencia con validez estadística

La media puede ser conveniente, pero no necesariamente válida. La regresión puede parecer sofisticada, pero también puede sobreajustar si los datos observados son pocos. KNN puede respetar vecindades, pero sufrir con alta dimensionalidad. No existe un método universalmente mejor; la elección depende del mecanismo de faltante, la geometría de los datos y el costo del error.

7. Aspectos numéricos y aparición adicional de NaN

7.1. NaN generados durante el preprocesamiento

Aun si el dataset original no tiene `NaN`, pueden generarse después. Algunos ejemplos clásicos son:

- normalización con desviación estándar cero;
- logaritmos de valores negativos o cero;
- divisiones por variables casi nulas;
- transformaciones de Box-Cox fuera de dominio;
- ratios construidos sobre denominadores faltantes.

7.2. Problemas en la función logística

La sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

es estable en teoría, pero numéricamente puede presentar overflow si $|z|$ es muy grande. Una implementación robusta usa estabilización numérica, truncamiento o rutinas de bibliotecas optimizadas. También puede usarse regularización para evitar coeficientes excesivos.

7.3. Separación perfecta

Si una variable o combinación lineal separa completamente las clases, la máxima verosimilitud clásica puede empujar ciertos coeficientes hacia magnitudes muy grandes. En términos prácticos, el optimizador converge mal o entrega parámetros extremos. Este problema es más probable cuando hay variables derivadas de imputación demasiado rígida o cuando el conjunto es pequeño.

7.4. Regularización como defensa

Una solución habitual es usar regularización L2 o L1:

$$J_\lambda(\boldsymbol{\beta}) = J(\boldsymbol{\beta}) + \lambda\|\boldsymbol{\beta}\|_2^2$$

o

$$J_\lambda(\boldsymbol{\beta}) = J(\boldsymbol{\beta}) + \lambda\|\boldsymbol{\beta}\|_1.$$

Esto mejora estabilidad, reduce magnitudes extremas y ayuda frente a colinealidad o sobreajuste.

8. Evaluación rigurosa del pipeline

8.1. Métricas de clasificación

En regresión logística, accuracy no debe ser la única métrica. Se recomienda complementar con:

- **Precision**: proporción de positivos predichos que realmente lo eran.
- **Recall**: proporción de positivos reales detectados.
- **F1-score**: balance entre precision y recall.
- **ROC-AUC**: capacidad de separación entre clases.
- **Log-loss**: calidad probabilística del modelo.

8.2. Validación cruzada dentro del pipeline

Cuando se comparan varias estrategias de imputación, la validación debe hacerse de manera pipeline-aware. Es decir, cada fold debe incluir ajuste del imputador y luego del modelo, no al revés. Esto evita optimismo artificial en resultados.

8.3. Uso de RMSE

Aunque la regresión logística es un clasificador, el RMSE puede aparecer de dos maneras útiles. Primero, para evaluar la calidad de un modelo auxiliar usado en imputación por regresión. Segundo, para comparar qué tan bien una estrategia reconstruye un valor continuo conocido en experimentos de borrado controlado. En ese sentido, el RMSE es una métrica del bloque de imputación, no necesariamente del clasificador binario final.

9. Cómo Monte Carlo puede complementar la solución

9.1. Motivación

Cuando existen datos faltantes, el valor imputado no es una verdad observada, sino una conjetura razonable. Por lo tanto, entrenar una sola vez con una sola imputación puede subestimar la incertidumbre real. Monte Carlo ofrece un marco para propagar esa incertidumbre hacia el modelo final.

9.2. Imputación múltiple estocástica

En lugar de imputar un valor único fijo, puede generarse una distribución de posibles valores plausibles. Por ejemplo, si una variable tiene media μ y desviación estándar σ, o si un modelo de regresión entrega una predicción $\hat{y}$ con residuales estimados, se pueden generar múltiples completaciones:

$$x_{\text{mis}}^{(b)} \sim p(x_{\text{mis}} \mid x_{\text{obs}}), \qquad b = 1, \ldots, B.$$

Para cada dataset imputado se ajusta una regresión logística distinta, obteniendo coeficientes y métricas diferentes. Esto permite estimar no solo un valor puntual sino un intervalo de incertidumbre.

9.3. Propagación de incertidumbre

Sea $\hat{\boldsymbol{\beta}}^{(b)}$ el vector de coeficientes del modelo entrenado con la imputación b. Entonces puede calcularse:

$$\bar{\boldsymbol{\beta}} = \frac{1}{B} \sum_{b=1}^{B} \hat{\boldsymbol{\beta}}^{(b)}$$

y también la variabilidad entre imputaciones:

$$\text{Var}_{\text{between}}(\hat{\boldsymbol{\beta}}) = \frac{1}{B-1} \sum_{b=1}^{B} \left(\hat{\boldsymbol{\beta}}^{(b)} - \bar{\boldsymbol{\beta}}\right) \left(\hat{\boldsymbol{\beta}}^{(b)} - \bar{\boldsymbol{\beta}}\right)^T.$$

Esto ofrece una visión mucho más honesta de la sensibilidad del modelo ante el faltante.

9.4. Simulación de escenarios

Monte Carlo también permite responder preguntas operativas como:

- ¿qué pasa con el recall si el porcentaje de faltantes sube de 5 % a 20 %?
- ¿qué tan estable es la frontera de decisión si el sensor crítico falla intermitentemente?
- ¿qué estrategia de imputación minimiza peor caso, no solo promedio?

Se pueden simular múltiples datasets sintéticos con distintos patrones de ausencia, entrenar el pipeline en cada uno y observar distribuciones de desempeño.

9.5. Monte Carlo para robustez de umbral

La regresión logística produce probabilidades y luego se aplica un umbral τ. Bajo incertidumbre en imputación, también puede analizarse la estabilidad del umbral óptimo. Monte Carlo permite estimar cuántas veces una observación cambia de clase al variar los valores faltantes dentro de rangos plausibles.

10. Ejemplo de pipeline en Python

Listing 1: Pipeline básico con imputación y regresión logística

```python
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(
        penalty="l2",
        C=1.0,
        max_iter=1000,
        solver="lbfgs"
    ))
])

pipe.fit(X_train, y_train)
```

```python

y_pred = pipe.predict(X_test)
y_prob = pipe.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_prob))
```

11. Ejemplo Monte Carlo para imputación múltiple

Listing 2: Esquema Monte Carlo simplificado para imputación múltiple

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

B = 100
auc_scores = []
coef_samples = []

for b in range(B):
    X_imp = X.copy()

    for j in range(X_imp.shape[1]):
        col = X_imp[:, j]
        mask = np.isnan(col)

        if np.any(mask):
            mu = np.nanmean(col)
            sigma = np.nanstd(col)

            # Imputacion estocastica simple
            col[mask] = np.random.normal(loc=mu, scale=sigma, size=mask
                .sum())
            X_imp[:, j] = col

    model = LogisticRegression(max_iter=1000)
    model.fit(X_imp_train, y_train)

    y_prob = model.predict_proba(X_imp_test)[:, 1]
    auc_scores.append(roc_auc_score(y_test, y_prob))
    coef_samples.append(model.coef_.ravel())

auc_scores = np.array(auc_scores)
coef_samples = np.array(coef_samples)

print("AUC media:", auc_scores.mean())
print("AUC std:", auc_scores.std())
```

```
print("Coeficientes medios:", coef_samples.mean(axis=0))
```

Este ejemplo es deliberadamente simple. En un entorno serio, conviene que la imputación estocástica se base en un modelo condicional más rico, no solo en muestreos marginales. Aun así, ilustra el principio: generar múltiples datasets plausibles, entrenar múltiples modelos y medir estabilidad.

12. Recomendaciones prácticas

12.1. Diagnóstico primero

Antes de imputar, medir porcentaje de faltantes por fila y por columna, identificar patrones temporales, correlaciones con otras variables y relación con la variable objetivo. Un mapa de calor de ausencia y una tabla por sensor suelen ser muy esclarecedores.

12.2. Reglas heurísticas razonables

Una guía práctica puede ser:

- si el faltante es bajo y la variable es estable, comenzar con mediana;
- si hay relaciones claras entre variables, probar regresión o imputación multivariante;
- si la estructura local importa, evaluar KNN;
- si una variable supera cierto umbral de ausencia, considerar eliminarla;
- agregar indicadores binarios de faltante cuando la ausencia pueda ser informativa;
- usar regularización en la regresión logística;
- validar siempre dentro de un pipeline.

12.3. Escenarios industriales

En sistemas con sensores, el faltante puede no ser ruido puro, sino síntoma de degradación, saturación o eventos operativos específicos. En tales casos, el patrón de ausencia debe modelarse como señal potencial, no solo como deficiencia a corregir.

13. Discusión

La regresión logística sigue siendo una herramienta poderosa incluso en presencia de datos faltantes, siempre que el tratamiento del faltante se aborde de manera rigurosa. Si se utiliza una

imputación coherente con la naturaleza del dato y se protege el pipeline contra fuga de información, el modelo puede conservar capacidad predictiva e interpretabilidad.

Sin embargo, el mayor riesgo no es técnico sino epistemológico: reemplazar un valor faltante por un número puede crear una falsa sensación de certeza. Desde ese punto de vista, Monte Carlo agrega valor no porque "mejore" mágicamente el clasificador, sino porque obliga a reconocer y cuantificar la incertidumbre inducida por la ausencia de datos. Esto resulta especialmente importante en decisiones donde el costo de un falso negativo o un falso positivo es alto.

14. Conclusiones

Los `NaN` surgen tanto por causas operativas como por errores numéricos y no deben tratarse como un detalle menor. En regresión logística, afectan entrenamiento, interpretación y estabilidad de parámetros. Su tratamiento exige comprender el mecanismo del faltante, seleccionar una estrategia de imputación adecuada y evaluar rigurosamente el impacto sobre el desempeño del clasificador.

La imputación por media o mediana ofrece baselines simples; la imputación por regresión y KNN explota estructura adicional; la eliminación selectiva evita sostener variables demasiado deterioradas. Más allá de ello, Monte Carlo permite incorporar el hecho fundamental de que un dato imputado no es un dato observado, sino una realización plausible dentro de un espacio de incertidumbre.

Por tanto, una solución robusta no consiste en un solo algoritmo aislado, sino en un pipeline compuesto por diagnóstico del faltante, imputación trazable, regresión logística regularizada, validación correcta y análisis de sensibilidad. Ese enfoque ofrece una base metodológica sólida para aplicaciones industriales, clínicas, financieras o académicas donde clasificar bien depende, antes que nada, de entender bien la calidad del dato.

Apéndice A. Fórmulas útiles

$$\pi(\boldsymbol{x}) = \frac{1}{1 + \exp[-(\beta_0 + \boldsymbol{\beta}^T \boldsymbol{x})]}$$

$$\log\left(\frac{\pi(\boldsymbol{x})}{1 - \pi(\boldsymbol{x})}\right) = \beta_0 + \boldsymbol{\beta}^T \boldsymbol{x}$$

$$J(\boldsymbol{\beta}) = -\sum_{i=1}^{n} [y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i)]$$

$$J_\lambda(\boldsymbol{\beta}) = J(\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_2^2$$

$$\text{Precision} = \frac{VP}{VP + FP}, \qquad \text{Recall} = \frac{VP}{VP + FN}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{ROC-AUC} = \int_0^1 TPR(FPR^{-1}(u))\, du$$

Apéndice B. Tabla comparativa de estrategias

Estrategia	**Ventaja principal**	**Riesgo principal**	**Cuándo usar**
Media	Muy simple y rápida	Reduce varianza, sensible a outliers	Baseline o faltante muy bajo
Mediana	Robusta a extremos	Puede perder relación multivariante	Variables sesgadas o con outliers
Regresión	Usa correlaciones entre variables	Supone estructura relativamente estable	MAR y covariables informativas
KNN	Flexible y local	Sensible a escala, costo computacional	Estructura no lineal local
Eliminar variable	Evita ruido excesivo	Puede perder señal útil	Faltante masivo o sensor degradado
Monte Carlo + imputación múltiple	Cuantifica incertidumbre	Más costo computacional y diseño	Decisiones críticas y análisis robusto